

TABUADA EM PYTHON – PASSO A PASSO COMPLETO

Vamos criar um programa que pede um número ao usuário e mostra a tabuada dele de 1 a 10.

1 INTRODUÇÃO

Configurar o Código da Tabuada

Vamos corrigir os erros no código para que ele funcione corretamente e mostre a tabuada de um número escolhido pelo usuário.



Meu objetivo é fazer o código funcionar corretamente!

2 O PROBLEMA

Código com Erros

```
#!/usr/bin/env python3
# Tabuada de um número
for i in range(1, 11):
    print(f'{num} x {i} = {num * i}')
```



Este código não funciona porque:

- A variável `num` não foi criada.
- O `input()` está incorreto.
- Faltou dois pontos `:` no `for`.
- A indentação do `print()` está errada.

3 PASSO 1

Pedir o número ao usuário



Use a função `input()` para perguntar o número e `int()` para transformar em número.

```
1 num = int(input("Quer ver a tabuada de qual número? "))
```



O que acontece aqui?

- `input()` mostra o prompt e espera o usuário digitar.
- O que o usuário digitar vem como texto (string).
- `int()` converte esse texto para número inteiro.
- O valor digitado é guardado na variável `num`.

Exemplo se o usuário digitar 5, então `num = 5`

4 PASSO 2

Colar o laço de repetição



Use o `for` para repetir de 1 a 10 (que são as linhas da tabuada).

```
1 for i in range(1, 11):
```

Entendendo cada parte:

- `for` → indica um laço de repetição.
- `i` → é a variável do controle (contador).
- `in` → significa "em" (percorre os valores).
- `range(1, 11)` → gera os números de 1 até 10 (o 11 não entra).
- `:` → dois pontos que indica o início do bloco.



Pelo os espaços de dois pontos `:` no final da linha.

5 PASSO 3

Imprimir a tabuada



Dentro do laço, usamos `print()` para mostrar a multiplicação.

```
1 print(f'{num} x {i} = {num * i}')
```

O que está acontecendo?

- `num` → número escolhido pelo usuário.
- `i` → o número da sequência (1 a 10).
- `num * i` → faz a multiplicação.
- `print()` → mostra o resultado na tela.



O `f` antes dos aspas permite usar as variáveis dentro do texto (f-string).

6 PASSO 4

Ajustar a indentação



Tudo que está dentro de `for` deve estar recuado (indentado).

```
1 for i in range(1, 11):
2     print(f'{num} x {i} = {num * i}')
```

← recuo (indentação)



Por que indentar?

A indentação mostra ao Python o que faz parte do laço. Sem indentação correta, o Python entende que o `print()` não faz parte do `for` e gera erro.

7 CÓDIGO FINAL CORRIGIDO

Funcionando corretamente

```
1 num = int(input("Quer ver a tabuada de qual número? "))
2
3 for i in range(1, 11):
4     print(f'{num} x {i} = {num * i}')
```

Exemplo de saída (número 5):

```
Quer ver a tabuada de qual número? 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
...
5 x 10 = 50
```

8 F-STRING EXPLICADO

O que é e como funciona

F-string (Formatted string) é uma forma moderna e prática de inserir variáveis dentro de textos.

```
print(f'{variavel1} texto {outra_variavel1}')
```

Como funciona?

1. Colocamos o texto f antes das aspas.
2. Usamos {} para colocar variáveis ou expressões.
3. O Python substitui os chaves pelo valor da variável.
4. Tudo virá uma única string textual para imprimir.

Mais exemplos:

```
• print(f'Nome: {nome}')
```

```
• print(f'Idade: {idade} e {g} = {num * g}')
```

```
• print(f'Resultado: {num * 10}')
```



Se quiseres códigos mais legíveis, utilize `br` e entre constantes strings `\"`.

9 RESUMO DO APRENDIZADO

O que é importante lembrar



`input()` → recebe dados do usuário.



`int()` → converte texto para número inteiro.



`for + range()` → repetição de ações.



Indentação → define os blocos do código.



`print()` → mostra informações na tela.



F-string → insere variáveis dentro do texto.



Seguindo esses passos, o código funciona e a tabuada é exibida corretamente!

COMO O PROGRAMA EXECUTA (PASSO A PASSO)



Pergunta ao usuário

Mostra o prompt e espera o usuário digitar um número.

```
Quer ver a tabuada de qual número? 5
```



Salva na variável

Converte o que foi digitado para número inteiro e armazena em `num`.

```
num = 5
```



Inicia o for

O `for` começa com `i = 1` e vai até `i = 10`.

```
1 = 1, 2, 3, ..., 10
```



Executa o bloco

Para cada valor de `i`, executa a multiplicação `i * print()`.

```
5 x 1 = 5
```



Repete até o fim

Vai repetindo até `i = 10` e então o programa termina.

```
5 x 10 = 50
```